UNIVERSITAS PAHLAWAN TUANKU TAMBUSAI
FAKULTAS TEKNIK — PROGRAM STUDI TEKNIK INFORMATIKA

# LEMBAR KERJA MAHASISWA

Pertemuan 6 · Pewarisan dan Method Khusus

| | | | |
|---|---|---|---|
| **Nama** | **Velia Arba Yales Tofa** | **NIM** | **2555201017** |
| **Kelas** | **A** | **Tanggal** | **05 November 2026** |
| **Tautan repositori** | **https://github.com/veliaarba/PBO-UP-2026.git** | | |

## A. KUIS AWAL — MENJEMBATANI DARI PERTEMUAN 5 (10 MENIT)

1. **Sebutkan uji pembeda antara komposisi dan agregasi.**
   Komposisi: Objek bagian mutlak bergantung pada pemiliknya dan ikut hancur jika pemiliknya dihapus. Agregasi: Objek bagian bersifat independen dan tetap hidup meskipun pemiliknya dihapus.

2. **Mengapa BarisSewa menyimpan salinan tarif, padahal ia sudah menyimpan rujukan ke alatnya?**
   Untuk menjaga konsistensi historis transaksi; agar jika tarif alat berubah di masa depan, tarif pada transaksi masa lalu tidak ikut berubah.

3. **Mengapa property baris mengembalikan tuple, bukan daftar?**
   Untuk menjaga enkapsulasi dan mencegah modifikasi eksternal (*immutability*).

4. **Tulis multiplisitas relasi Sewa – BarisSewa di kedua ujungnya.**
   Sewa ke BarisSewa: 1..* (atau 1), BarisSewa ke Sewa: 1.

# B. PENGAMATAN PRAKTIKUM

## B.1 super() yang terlupa (Langkah 3)

| Pertanyaan | Jawaban Anda |
|---|---|
| Apakah objeknya berhasil dibuat? Apakah ada galat saat itu? | Objek berhasil dibuat saat instansiasi, tetapi terjadi galat (error) ketika method turunan dipanggil. |
| Apa isi vars(t), dan berapa atribut yang seharusnya ada? | Isi vars(t) hanya berisi atribut milik kelas turunan saja (misalnya _tenaga_kuda), dan hanya ada 1 atribut (atribut induk seperti _kode, _nama, dan _tarif_harian tidak ada karena super().__init__() terlewat). |
| Method mana yang runtuh, dan atribut apa yang dicarinya? | Method biaya_sewa() runtuh karena mencoba mencari atribut _tarif_harian milik kelas induk yang gagal diinisialisasi. |
| Dengan kalimat sendiri: apa sesungguhnya yang dikerjakan super().__init__()? | Memanggil konstruktor kelas induk untuk menginisialisasi atribut dasar milik induk ke dalam objek turunan. |

## B.2 __eq__ tanpa __hash__ (Langkah 7)

| Pertanyaan | Jawaban Anda |
|---|---|
| Berapa nilai AlatTani.__hash__yang ditampilkan? | None (atau memunculkan galat bahwa objek tidak bisa di-hash / unhashable type). |
| Tiga tindakan apa yang gagal, dan apa pesan galatnya? | • Memasukkan objek ke dalam set() -> Pesan galat: TypeError: unhashable type: 'AlatTani'<br>• Menjadikan objek sebagai kunci dalam dict -> Pesan galat: TypeError: unhashable type: 'AlatTani'<br>• Memanggil fungsi hash() pada objek -> Pesan galat: TypeError: unhashable type: 'AlatTani' |
| Mengapa Python mematikan __hash__ begitu __eq__ditulis ulang? | Karena Python menjaga konsistensi antara kesetaraan dan *hash*. Jika dua objek dianggap sama (__eq__ bernilai True), keduanya wajib menghasilkan nilai *hash* yang sama. Ketika __eq__ diubah kustom, Python tidak tahu lagi cara memastikan aturan ini, sehingga secara otomatis membuat objek menjadi *unhashable*. |
| Aturan apa yang harus dipenuhi __eq__ dan __hash__agar konsisten? | Jika dua objek bernilai sama menurut __eq__ (mengembalikan True), maka kedua objek tersebut **wajib** memiliki nilai hash yang sama persis. |

## B.3 Dua cara perkakas menangkap kesalahan

| Berkas | Kode Aturan Ruff | Menunjuk Gejala atau Penyakit? |
|---|---|---|
| `lupa_super.py` | F841 / ECO... (atau error terkait atribut tidak terdefinisi/inisialisasi tidak lengkap) | Menunjuk **Penyakit** (akar masalah kegagalan konstruktor induk). |
| `eq_tanpa_hash.py` | RUF012 / B032 (atau aturan terkait *unhashable type* / definisi __eq__ tanpa __hash__) | Menunjuk **Penyakit** (cacat desain struktural pada logika kesetaraan objek). |

**1. Ruff menangkap keduanya, tetapi dengan cara yang berbeda. Jelaskan perbedaannya, dan apa artinya bagi cara Anda membaca peringatan perkakas.**

Perbedaan cara Ruff menangkap:

- lupa_super.py (Struktural/Statis): Ruff mendeteksi lewat analisis AST (*Abstract Syntax Tree*) bahwa konstruktor kelas turunan tidak memanggil super().__init__(), sehingga atribut dasar tidak pernah diinisialisasi.
- eq_tanpa_hash.py (Kontrak Objek): Ruff mendeteksi pelanggaran aturan kelas (melalui aturan spesifik seperti B023 / RUF016) karena kelas mendefinisikan __eq__ tanpa mendefinisikan __hash__, yang melanggar kontrak internal Python untuk struktur data berbasis *hash*.

Artinya bagi cara membaca peringatan: Peringatan perkakas (*linter*) bukan sekadar gangguan atau formalitas sintaks, melainkan diagnostik akar masalah (penyakit struktural) pada desain kode PBO kita sebelum kode tersebut dijalankan atau runtuh saat diuji.

## C. AKTIVITAS KELOMPOK: KAPAN PEWARISAN MERUGIKAN

Anggota kelompok: Velia Arba Yales Tofa, Hikmatul Mardhatillah

### C.1 Uji "adalah sejenis" pada proyek kelompok

| Pasangan Kelas | Kalimat "A adalah sejenis B" | Masuk Akal? Relasi yang Tepat? |
|---|---|---|
| Traktor – AlatTani | Traktor adalah sejenis AlatTani. | Masuk akal. Menggunakan inheritance karena Traktor merupakan jenis alat pertanian. |
| PompaAir – AlatTani | PompaAir adalah sejenis AlatTani. | Masuk akal. Menggunakan inheritance karena PompaAir termasuk alat pertanian. |
| Perontok – AlatTani | Perontok adalah sejenis AlatTani. | Masuk akal. Menggunakan inheritance karena Perontok merupakan jenis alat pertanian. |

### C.2 Kemiripan yang sebaiknya diselesaikan dengan komposisi

Sebutkan satu, beserta alasannya. Butir ini bernilai paling tinggi.

Kelas Sewa dan BarisSewa

Hubungan antara kelas Sewa dan BarisSewa lebih tepat menggunakan komposisi daripada pewarisan. Hal ini karena satu transaksi penyewaan dapat terdiri dari beberapa BarisSewa yang berisi informasi alat pertanian, jumlah, dan biaya penyewaan. BarisSewa merupakan bagian dari transaksi Sewa, bukan jenis dari Sewa. Oleh karena itu, kedua kelas tersebut tidak perlu menggunakan inheritance. Komposisi lebih sesuai karena BarisSewa menjadi bagian dari data transaksi penyewaan.

## D. REFLEKSI PENUTUP

**1. Validasi tarif yang Anda tulis di Pertemuan 3 kini menjaga tiga kelas baru tanpa satu baris pun disalin. Menurut Anda apa risikonya bila suatu hari validasi itu perlu berbeda untuk salah satu turunan?**

Jika aturan validasi (misalnya batas minimal tarif) diletakkan secara kaku di kelas induk AlatTani lalu suatu hari salah satu kelas turunan butuh aturan validasi yang berbeda, maka kelas turunan tersebut terpaksa melanggar prinsip pewarisan (*Liskov Substitution Principle*), atau kita harus merusak enkapsulasi dengan menambahkan pengecualian khusus (if/else) yang kotor di dalam kelas induk.

**2. Sebutkan satu hal yang belum Anda pahami, dan pertanyaan untuk Pertemuan 7.**

Bagaimana cara merancang antarmuka (*interface* atau kelas abstrak) yang benar-benar bersih ketika kelas-kelas turunan di bawahnya memiliki atribut spesifik yang sangat bervariasi satu sama lain?

| Paraf Asisten / Dosen | Tanda Tangan Mahasiswa |
|---|---|
| ..... | Velia Arba Yales Tofa |